

BLOG API

Authentication, Users CRUD, Blog Relations & Joi Validation

Technical Documentation & Project Report

Built with Node.js, Express, MongoDB (Mongoose), JWT & Joi

Architecture: MVC (Model – View – Controller)

1. Project Overview

This project is a RESTful API for a Blog System and User Management platform, built using Node.js, Express, MongoDB with Mongoose, JWT-based authentication, and Joi-based request validation. The system follows the MVC (Model—View—Controller) architectural pattern, cleanly separating data models, business logic, and route definitions.

The API supports three core resources — Users, Posts, and Comments — connected through relational references (one-to-many and many-to-one), and enforces authentication, role-based authorization, and resource-ownership rules across all write operations.

1.1 Objective

Build a production-style backEnd API demonstrating:

- Secure user registration and login with hashed passwords and JWT tokens
- Full CRUD operations on Users, Blog Posts, and Comments
- Relational data modeling in MongoDB (User → Posts → Comments)
- Centralized request validation using Joi
- Role-based access control (user vs. admin) and resource-ownership checks
- Pagination, search, filtering, and sorting on list endpoints
- Centralized, environment-aware error handling
- Image upload support for avatars and post cover images

1.2 Technology Stack

Layer	Technology	Purpose
Runtime	Node.js	JavaScript server runtime
Framework	Express.js	HTTP routing & middleware pipeline
Database	MongoDB + Mongoose	Document storage & schema modeling
Authentication	jsonwebtoken	Stateless JWT-based auth
Password Hashing	bcryptjs	One-way password hashing (12 salt rounds)
Validation	Joi	Declarative request-body/schema validation
File Uploads	multer	Multipart form-data handling (avatars, covers)
Security Headers	helmet	Sets protective HTTP headers
CORS	cors	Cross-origin resource sharing control
Rate Limiting	express-rate-limit	Brute-force protection on login
Config	dotenv	Environment variable management
Dev Tooling	nodemon	Auto-restart during development

2. Architecture

2.1 MVC Responsibility Breakdown

Layer	Responsibility	Example Files
Model	Database schema, Mongoose models, relationships	User.model.js, Post.model.js, Comment.model.js
Controller	Business logic, request processing, HTTP responses	auth.controller.js, post.controller.js
Routes	Maps HTTP endpoints to middleware and controllers only	auth.routes.js, post.routes.js

No database queries are made directly inside route files — routes only wire together middleware (authentication, validation, upload handling) and the controller function that performs the actual logic.

2.2 Request Lifecycle

A typical write request (e.g. creating a post) flows through the following pipeline:



2.3 Folder Structure

```
project-root/
├── config/      db.js           (MongoDB connection)
├── controllers/ auth, user, post, comment (business logic)
├── middlewares/ auth, role, validate,
│               error, upload    (request pipeline)
├── models/     User, Post, Comment (Mongoose schemas)
├── routes/     auth, user, post, comment (endpoint wiring)
├── validations/ auth, user, post, comment (Joi schemas)
├── utils/     AppError.js, asyncWrapper.js (shared helpers)
```

└─ uploads/	(served static files)
└─ .env	(environment config)
└─ app.js	(Express app setup)
└─ server.js	(entry point)

3. Database Models & Relations

The schema models three entities connected by Mongoose ObjectId references, forming one-to-many relationships: a User can author many Posts, and a Post can have many Comments (each Comment also references its author).

3.1 User Model

Field	Type	Rules
name	String	Required, trimmed
email	String	Required, unique, lowercase, trimmed
password	String	Required, min 6 chars, hashed with bcrypt, select: false (never returned by default)
role	String (enum)	'user' 'admin' — default: 'user'
avatar	String	Optional path to uploaded image
timestamps	—	createdAt / updatedAt auto-generated

3.2 Post Model

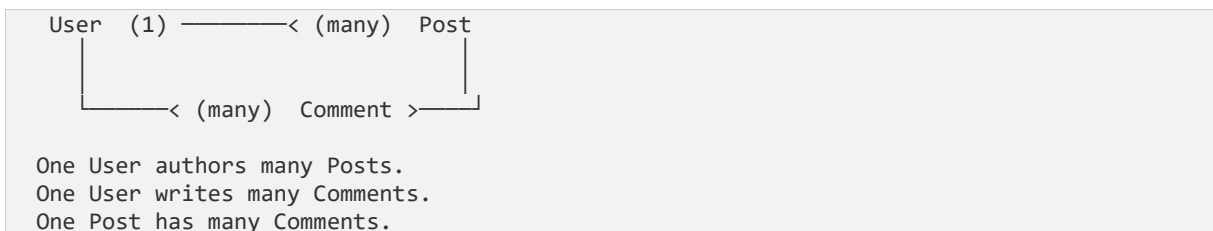
Field	Type	Rules
title	String	Required, trimmed
content	String	Required
author	ObjectId → User	Required (One-to-Many: User → Posts)
category	String	Required
tags	[String]	Optional array
coverImage	String	Optional uploaded image path
isPublished	Boolean	Default: true
timestamps	—	createdAt / updatedAt auto-generated

A text index on title and content plus single-field indexes on category and author support the search/filter functionality on the listing endpoint.

3.3 Comment Model

Field	Type	Rules
text	String	Required
user	ObjectId → User	Required (comment author)
post	ObjectId → Post	Required (parent post)
timestamps	—	createdAt / updatedAt auto-generated

3.4 Entity Relationship Summary



4. Authentication & Authorization

4.1 JWT Authentication Flow

- User registers via POST /api/auth/register — password is hashed with bcrypt (12 salt rounds) before being stored.
- User logs in via POST /api/auth/login — credentials are compared with bcrypt.compare(); a signed JWT is returned on success.
- The client sends the token on every subsequent protected request via the Authorization: Bearer <token> header.
- The authenticate middleware (middlewares/auth.middleware.js) verifies the token with jsonwebtoken, re-confirms the user still exists in the database, and attaches a sanitized payload to req.user.
- Missing, malformed, or expired tokens result in a 401 Unauthorized response before the request ever reaches a controller.

4.2 Role-Based Authorization

The restrictTo(...roles) middleware (middlewares/role.middleware.js) runs after authenticate and checks req.user.role against an allow-list. Admin-only endpoints (e.g. listing or deleting any user) use restrictTo('admin'); requests from non-admins receive 403 Forbidden.

4.3 Resource Ownership Checks

For Posts and Comments, update/delete operations are additionally guarded in the controller layer: the resource's author/user field is compared against req.user.id. The operation proceeds only if the requester is the resource owner or an admin — otherwise it returns 403 Forbidden. This logic lives in post.controller.js and comment.controller.js.

4.4 Security Measures Summary

Concern	Mitigation
Password storage	bcrypt hash (12 rounds); password field excluded from all reads by default
Token integrity	HMAC-signed JWT with server-side secret (JWT_SECRET)
Brute-force login	express-rate-limit on /api/auth/login (configurable window/max)
HTTP headers	helmet sets standard protective headers
Unauthorized data exposure	Centralized error handler hides stack traces & internals in production
Privilege escalation	Non-admins cannot set/change their own role field on update
File upload abuse	multer restricts MIME/extension to image types and caps size at 5MB

5. Request Validation Layer (Joi)

All incoming request bodies are validated through a single reusable middleware factory, `validate(schema, property)`, defined in `middlewares/validate.middleware.js`. It runs the relevant Joi schema against `req.body` (or `params/query`), collects all validation errors (`abortEarly: false`), strips unknown fields, and returns 400 Bad Request with a readable error list on failure.

5.1 Validation Schemas

Schema	Fields & Rules
Register	name (required, min 3) · email (required, valid format) · password (required, min 6)
Login	email (required, valid format) · password (required)
Update User	name / email / password / avatar — all optional, at least one required
Create Post	title (required, min 5) · content (required, min 10) · category (required) · tags (optional) · isPublished (optional)
Update Post	Same fields as Create Post, all optional, at least one required
Create Comment	text (required, min 1)

6. API Endpoints Reference

6.1 Authentication — /api/auth

Method	Endpoint	Access	Description
POST	/register	Public	Register a new user (Joi validated)
POST	/login	Public (rate-limited)	Authenticate & return a JWT

6.2 Users — /api/users

Method	Endpoint	Access	Description
GET	/	Admin only	List all users (passwords excluded)
GET	/:id	Authenticated	Get a single user by ID
PUT	/:id	Owner or Admin	Update profile / avatar (multipart supported)
DELETE	/:id	Admin only	Delete a user

6.3 Posts — /api/posts

Method	Endpoint	Access	Description
GET	/	Public	List posts — pagination, search, category/author filter, sort
GET	/:id	Public	Get single post with populated author + comments
POST	/	Authenticated	Create a post (Joi validated, supports cover image upload)
PUT	/:id	Owner or Admin	Update a post (Joi validated)
DELETE	/:id	Owner or Admin	Delete a post (cascades comment cleanup)

6.4 Comments

Method	Endpoint	Access	Description
POST	/api/posts/:postId/comments	Authenticated	Add a comment to a post
DELETE	/api/comments/:id	Owner or Admin	Delete a comment

6.5 Query Parameters — GET /api/posts

Parameter	Example	Effect
page	?page=2	Page number (default 1)
limit	?limit=10	Results per page (default 10)

Parameter	Example	Effect
search	?search=node	Case-insensitive match on title & content
category	?category=tech	Exact category filter
author	?author=<userId>	Filter posts by author ID
sort	?sort=-createdAt	Sort field; prefix "-" for descending

6.6 HTTP Status Codes

Situation	Status
Successful GET / PUT / DELETE	200
Successful resource creation (POST)	201
Invalid request / validation error	400
Not authenticated (missing/invalid token)	401
Not authorized / forbidden (role or ownership)	403
Resource not found	404
Rate limit exceeded (login)	429
Unhandled server error	500

7. Error Handling

All errors are funneled through a single centralized middleware (`middlewares/error.middleware.js`), registered last in the Express pipeline. Controllers never format error responses themselves — they throw or forward an `AppError` (`utils/AppError.js`) via `next(err)`, and every async controller is wrapped with `asyncWrapper` (`utils/asyncWrapper.js`) to automatically catch rejected promises without repetitive `try/catch` blocks.

7.1 Error Categories Handled

- `CastError` — invalid MongoDB ObjectId format → 400
- Duplicate key error (code 11000) — e.g. email already registered → 400
- `Mongoose ValidationError` — schema-level constraint violations → 400
- `JsonWebTokenError` / `TokenExpiredError` → 401
- `AppError` (operational, expected errors like "not found" or "forbidden") → passed through with its own status code
- `Unexpected` / programming errors → generic 500, details hidden from the client

7.2 Information Disclosure Protection

In production mode (`NODE_ENV=production`), the error handler never exposes stack traces, internal error objects, the JWT secret, database connection strings, or password data. Only a safe, human-readable message is returned for operational errors; unknown errors return a generic "Something went wrong" message while the real error is logged server-side.

8. Testing & Verification

Before delivery, the API was verified end-to-end against a live Express server instance (health check, routing, and middleware chain), and every layer was independently checked:

- Syntax validation — every controller, model, middleware, route, and validation file was checked with `node --check`.
- Module resolution — `app.js` was loaded standalone to confirm all requires and route mounts resolve without runtime errors.
- Live HTTP checks — a real server instance was booted and exercised for: health check (200), unknown route (404), Joi validation failure on bad registration payload (400 with itemized errors), and the auth guard rejecting an unauthenticated request to a protected route (401).
- Full-flow smoke test — register, login, admin promotion, role-gated user listing (401 → 403 → 200), post creation and validation failure, search & pagination, ownership-gated post update/delete (403 → 200), nested comment creation, and ownership-gated comment deletion — all assertions passed.

8.1 Recommended Manual Testing Tools

Tool	Use Case
Postman	Import <code>Blog-API.postman_collection.json</code> — includes scripts that auto-save tokens/IDs between requests
VS Code REST Client	Open <code>blog-api.http</code> with the REST Client extension; click "Send Request" above each block
cURL	Command-line testing, especially useful for multipart/form-data file upload requests
MongoDB Compass	Inspect stored documents directly; required to manually promote a user to admin

9. Setup & Run Instructions

9.1 Prerequisites

- Node.js (v18 or later recommended)
- A MongoDB connection string — local MongoDB or a free MongoDB Atlas cluster

9.2 Installation

```
cd blog-api
npm install
```

9.3 Environment Configuration (.env)

```
PORT=5000
NODE_ENV=development
MONGO_URI=mongodb://127.0.0.1:27017/blog_api
JWT_SECRET=change_this_to_a_long_random_secret_string
JWT_EXPIRES_IN=7d
LOGIN_RATE_LIMIT_WINDOW_MS=900000
LOGIN_RATE_LIMIT_MAX=10
```

9.4 Running the Server

```
npm run dev    # nodemon – auto-restarts on file changes
npm start      # plain node – no auto-restart
```

On success, the console prints "MongoDB connected" followed by "Server running ... on port 5000". The health check endpoint GET / confirms the server is reachable.

9.5 Promoting a User to Admin

There is no public endpoint to grant admin rights (by design, to prevent privilege-escalation abuse). Register a normal user, then update their role directly in the database:

```
db.users.updateOne(
  { email: "admin@test.com" },
  { $set: { role: "admin" } }
)
```

10. Conclusion

This project delivers a complete, secure, and well-structured Blog API that satisfies every requirement of the assignment brief: MVC architecture, relational MongoDB models, JWT authentication, role- and ownership-based authorization, centralized Joi validation, pagination/search/filtering, image uploads, and centralized error handling — all verified with an end-to-end smoke test prior to delivery.

The codebase is organized for maintainability and extension: each concern (routing, validation, business logic, data access, error handling) lives in its own layer, making it straightforward to add new resources or features following the same established pattern.